

# Práctico Final

---

## Objetivo

---

Construir una aplicación full-stack integrando un frontend Angular con el backend back (NestJS). El trabajo se divide en dos niveles: **aprobar** (funcionalidades de verificación de email y recuperación de contraseña) y **promocionar** (servidor MCP que exponga la API como herramientas).

El backend back ya está desarrollado en `back/` (NestJS + PostgreSQL) y se entrega funcionando en `http://localhost:3000`. Deben **extenderlo** agregando los módulos de verificación de email y recuperación de contraseña.

El frontend Angular está en `front/` con la estructura base (login, register, profile, navbar, guards, interceptores). Deben modificarlo para integrar el flujo completo de verificación y recuperación.

---

## Punto de partida

---

### Requisitos

- Node.js
- Servicio de email (Resend, SMTP real, etc.)

### Para arrancar

```
# Backend
cd back && npm install && npm run start:dev

# Frontend (otra terminal)
cd front && npm install && npm start
```

---

## 1. Condiciones para Aprobar

---

### 1.1 Verificación de email — Backend

Modificar `UserEntity` — agregar:

- `isVerified` booleano default `false`

- `verificationToken` string nullable

Modificar `POST /auth/register` :

- Después de crear el usuario, generar un token único y guardarlo
- Enviar un email con un link de verificación que incluya el token
- **No** devolver el token en la response

Crear `POST /auth/verify-email` (sin auth):

- Body: `{ token }`
- Buscar user por token, marcar `isVerified = true` , limpiar token
- Responder `{ message: "Email verificado" }`
- **Response 400** : `{ "message": "Token inválido o expirado" }`

El link enviado por email debe apuntar a: `http://localhost:4200/verify-email?token=<token>`

Crear `POST /auth/resend-verification` (JWT):

- Generar nuevo token, guardarlo, reenviar el email

Modificar `GET /auth/me` — incluir `isVerified` en la response.

## 1.2 Verificación de email — Frontend

Crear `/verify-pending` (sin auth):

- "Revisá tu email. Te enviamos un link de verificación."
- Botón "Reenviar email" → llama a `resend-verification`

Crear `/verify-email?token=xxx` (sin auth):

- Leer token de `queryParams`, llamar a `verify-email`
- Mostrar "Email verificado correctamente"
- Link a `/login`
- Si el token es inválido: mostrar error y link para reenviar email (reenvío requiere JWT, el usuario debe iniciar sesión)

Modificar `register.ts` :

- Después del registro exitoso, redirigir a `/verify-pending`

Modificar `profile.html` :

- Badge "Verificado" o "No verificado"
- Si no está verificado, botón "Reenviar email"

## 1.3 Recuperación de contraseña — Backend

Agregar a `UserEntity` :

- `resetPasswordToken` string nullable
- `resetPasswordExpires` datetime nullable

Crear `POST /auth/forgot-password` (sin auth):

- Body: `{ email }`
- Si el email existe: generar token + expiry (ej. 1 hora), guardar, enviar email con link de reset
- Responder siempre el mismo mensaje (seguridad): `{ message: "Si el email existe, recibirás un link" }`

El link enviado por email debe apuntar a: `http://localhost:4200/reset-password?token=<token>`

Crear `POST /auth/reset-password` (sin auth):

- Body: `{ token, password }`
- Buscar user por token, validar que no haya expirado
- Hashear nueva contraseña, actualizar, limpiar campos de reset
- Responder `{ message: "Contraseña actualizada" }`
- **Response 400** : `{ "message": "Token inválido o expirado" }`

## 1.4 Recuperación de contraseña — Frontend

Crear `/forgot-password` (sin auth):

- Input email, botón "Enviar link"
- Mostrar mensaje genérico "Si el email existe..."

Crear `/reset-password?token=xxx` (sin auth):

- Leer token de `queryParams`
- Inputs: nueva contraseña + confirmar (mín 8 chars)
- Validar que coincidan
- Llamar a `reset-password`
- Mostrar éxito con link a `/login`

Modificar `login.html` :

- Link "¿Olvidaste tu contraseña?" → `/forgot-password`

## 1.5 Toast / notificaciones

## 1.5 Tests, Mocks y Mocking

Crear `services/toast.service.ts` y `shared/toast/` component:

- Servicio con métodos `success()`, `error()`, `info()`
- Componente que se muestra y se auto-destruye a los segundos
- Usar toasts en lugar de `{{ error }}` local en:
  - `register.ts` → "Revisá tu email. Te enviamos un link de verificación"
  - `verify-email.ts` → "Email verificado correctamente" / "Token inválido o expirado"
  - `resend-verification` → "Email reenviado"
  - `forgot-password.ts` → "Si el email existe, recibirás un link"
  - `reset-password.ts` → "Contraseña actualizada"
  - `login.ts` → errores de credenciales inválidas

## 1.6 Perfil — cambiar contraseña y email

Modificar `profile.html` / `profile.ts`:

- Badge "Verificado" / "No verificado"
- Formulario "Cambiar contraseña": contraseña actual, nueva contraseña, confirmar → llama a `PATCH /users/me/password`
- Formulario "Cambiar email": nuevo email + contraseña actual → llama a `PATCH /users/me/email`
- Mostrar toast en éxito/error de cada operación

## 1.7 Criterios de evaluación

- El email de verificación se envía correctamente al registrarse
  - El link de verificación funciona y marca al usuario como verificado
  - El reenvío de verificación funciona
  - El formulario de forgot-password envía el email de recuperación
  - El link de reset-password permite cambiar la contraseña
  - Todos los errores se muestran al usuario en pantalla (nunca en consola)
  - El frontend refleja correctamente el estado de verificación
  - Los endpoints usan `class-validator` para validación
  - Los tokens se generan con `crypto.randomUUID()`
  - Los toasts se muestran correctamente en todos los flujos del práctico
  - Se puede cambiar la contraseña desde el perfil
  - Se puede cambiar el email desde el perfil
  - El email se envía con un servicio real (Resend, SendGrid, SMTP, etc.), no se aceptan mocks ni console.log
-

## 2. Condiciones para Promocionar

### 2.1 Objetivo

Construir un servidor MCP (Model Context Protocol) que exponga los endpoints de la API REST `back` como herramientas (tools) invocables desde opencode.

Se te proporciona un **template base** con las herramientas de autenticación ya implementadas. Tu tarea es completar el resto de las tools.

### 2.2 Template base

El template incluye:

| Archivo                              | Propósito                                                                                                                   |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| <code>mcp/src/index.ts</code>        | Punto de entrada del servidor MCP                                                                                           |
| <code>mcp/src/api-client.ts</code>   | Cliente HTTP para comunicarse con back                                                                                      |
| <code>mcp/src/tool-factory.ts</code> | Utilidad para registrar tools en el servidor                                                                                |
| <code>mcp/src/tools/auth.ts</code>   | Tools de auth: <code>auth_login</code> , <code>auth_register</code> , <code>auth_me</code> , <code>delete_my_account</code> |
| <code>mcp/src/tools/index.ts</code>  | Índice que agrupa todas las tools                                                                                           |

### Estructura de una tool

Cada tool se define como un objeto con la siguiente forma:

```
{
  name: "nombre_de_la_tool",
  description: "Descripción clara de lo que hace",
  inputSchema: {
    param1: z.string(),
    param2: z.number().int().optional(),
  },
  handler: async (args) => api.get("/ruta", args),
}
```

Usar `zod` para definir los esquemas de validación de entrada y `api` (de `api-client.ts`) para hacer las requests HTTP.

Consultar `tools/auth.ts` como ejemplo de implementación.



## 2.3 Endpoints a cubrir

### Products (archivo: `mcp/src/tools/products.ts`)

| Tool                        | Endpoint                          | Auth      | Descripción                     |
|-----------------------------|-----------------------------------|-----------|---------------------------------|
| <code>list_products</code>  | <code>GET /products</code>        | JWT       | Listar con filtros y paginación |
| <code>get_product</code>    | <code>GET /products/:id</code>    | JWT       | Obtener uno                     |
| <code>create_product</code> | <code>POST /products</code>       | JWT+Admin | Crear                           |
| <code>update_product</code> | <code>PUT /products/:id</code>    | JWT+Admin | Actualizar                      |
| <code>delete_product</code> | <code>DELETE /products/:id</code> | JWT+Admin | Eliminar                        |

### Categories (archivo: `mcp/src/tools/categories.ts`)

| Tool                         | Endpoint                            | Auth      | Descripción  |
|------------------------------|-------------------------------------|-----------|--------------|
| <code>list_categories</code> | <code>GET /categories</code>        | JWT       | Listar todas |
| <code>get_category</code>    | <code>GET /categories/:id</code>    | JWT       | Obtener una  |
| <code>create_category</code> | <code>POST /categories</code>       | JWT+Admin | Crear        |
| <code>update_category</code> | <code>PUT /categories/:id</code>    | JWT+Admin | Actualizar   |
| <code>delete_category</code> | <code>DELETE /categories/:id</code> | JWT+Admin | Eliminar     |

### Users (archivo: `mcp/src/tools/users.ts`)

| Tool                            | Endpoint                              | Auth      | Descripción        |
|---------------------------------|---------------------------------------|-----------|--------------------|
| <code>list_users</code>         | <code>GET /users</code>               | JWT+Admin | Listar usuarios    |
| <code>update_user_role</code>   | <code>PATCH /users/:id/role</code>    | JWT+Admin | Cambiar rol        |
| <code>update_my_password</code> | <code>PATCH /users/me/password</code> | JWT       | Cambiar contraseña |
| <code>update_my_email</code>    | <code>PATCH /users/me/email</code>    | JWT       | Cambiar email      |

### Actualizar índice

Una vez creados los archivos, importarlos y agregarlos al array en `mcp/src/tools/index.ts`.

## 2.4 Dependencias

El proyecto ya tiene `package.json` con las dependencias necesarias. Solo ejecutar:

```
cd mcp && npm install
```

## 2.5 Configuración en opencode

Para que opencode use tu servidor MCP, debe estar declarado en `opencode.json`:

```
{
  "mcp": {
    "back": {
      "type": "local",
      "command": ["npx", "tsx", "mcp/src/index.ts"],
      "enabled": true,
      "environment": {
        "API_C_URL": "http://localhost:3000",
        "API_C_EMAIL": "admin@mail.com",
        "API_C_PASSWORD": "12345678"
      }
    }
  }
}
```

`API_C_EMAIL` y `API_C_PASSWORD` son opcionales (se puede llamar `auth_login` primero).

## 2.6 Verificación

Después de implementar, probar que las herramientas aparecen en opencode con:

```
npx tsx mcp/src/index.ts
```

Y en opencode deberías poder ejecutar herramientas como `list_products`, `create_category`, etc.

## 2.7 Criterios de evaluación

- Todas las tools implementadas correctamente
  - Schemas de validación con zod (tipos correctos, campos requeridos vs opcionales)
  - Manejo de errores (el `tool-factory.ts` ya captura errores de la API)
  - El servidor MCP se inicia sin errores
  - opencode puede llamar exitosamente cada tool
-

## 3. Entrega

---

- **Nombre del archivo:** `desarrollo-N.zip` (N = número de grupo)
- **Formato:** ZIP (sin `node_modules/` , sin `.env` , sin `dist/` )
- **Estructura:**

```
desarrollo-N.zip
├── back/           (back completo con modificaciones)
├── front/          (frontend completo con modificaciones)
└── mcp/           (solo si promociona)
```

- **Todos los grupos deben completar la sección 1 (Aprobar)**
- **Para promocionar:** debe estar completo el nivel aprobar + sección 2

---

## 4. Referencia de la API

---

### Convenciones

- **JWT:** requiere header `Authorization: Bearer <token>`
- **Admin:** requiere JWT + rol `admin`
- Todos los cuerpos son JSON ( `Content-Type: application/json` )

---

### 4.1 Auth

#### **POST /auth/register**

Registra un nuevo usuario. El primer usuario registrado obtiene rol `admin` , los siguientes `user` .

**Request body:**

| Campo    | Tipo   | Obligatorio            |
|----------|--------|------------------------|
| email    | string | Sí (email válido)      |
| password | string | Sí (mín. 8 caracteres) |

```
{
  "email": "user@mail.com",
  "password": "12345678"
}
```



## Response 201 :

```
{
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "user@mail.com",
    "role": "user",
    "createdAt": "2026-06-07T21:00:00.000Z"
  },
  "access_token": "eyJhbGciOiJI..."
}
```

Protegido: **No**

---

## POST /auth/login

Inicia sesión con credenciales existentes.

### Request body:

| Campo    | Tipo   | Obligatorio |
|----------|--------|-------------|
| email    | string | Sí          |
| password | string | Sí          |

```
{
  "email": "user@mail.com",
  "password": "12345678"
}
```

## Response 201 :

```
{
  "user": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "email": "user@mail.com",
    "role": "user",
    "createdAt": "2026-06-07T21:00:00.000Z"
  },
  "access_token": "eyJhbGciOiJI..."
}
```

Protegido: **No**

---

## GET /auth/me

Devuelve el usuario autenticado actual.

Response 200 :

```
{
  "id": "550e8400-e29b-41d4-a716-446655440000",
  "email": "user@mail.com",
  "role": "user",
  "createdAt": "2026-06-07T21:00:00.000Z"
}
```

Protegido: JWT

---

## 4.2 Products

### GET /products

Lista productos con filtros opcionales y paginación.

Query params (todos opcionales):

| Parámetro | Tipo   | Descripción                                                                                   |
|-----------|--------|-----------------------------------------------------------------------------------------------|
| name      | string | Filtro por nombre (búsqueda parcial)                                                          |
| sortBy    | string | Campo de orden: <code>id</code> , <code>name</code> , <code>price</code> , <code>stock</code> |
| order     | string | Dirección: <code>ASC</code> o <code>DESC</code>                                               |
| page      | number | Número de página ( $\geq 1$ )                                                                 |
| limit     | number | Elementos por página (1–100)                                                                  |

Response 200 :

```
{
  "items": [
    {
      "id": 1,
      "name": "Producto Ejemplo",
      "price": 150.50,
      "stock": 10,
      "categoryId": 1,
      "category": { "id": 1, "name": "Electrónica" }
    }
  ],
}
```

```
"total": 1,  
"page": 1,  
"limit": 10  
}
```

Protegido: JWT

---

## GET /products/:id

Obtiene un producto por su ID.

**Response 200 :**

```
{  
  "id": 1,  
  "name": "Producto Ejemplo",  
  "price": 150.50,  
  "stock": 10,  
  "categoryId": 1,  
  "category": { "id": 1, "name": "Electrónica" }  
}
```

**Response 404 :** "Product not found"

Protegido: JWT

---

## POST /products

Crea un nuevo producto.

**Request body:**

| Campo      | Tipo          | Obligatorio                |
|------------|---------------|----------------------------|
| name       | string        | Sí (máx. 256 caracteres)   |
| price      | number        | Sí (> 0, máx. 4 decimales) |
| stock      | number        | No (default 0, ≥ 0)        |
| categoryId | number   null | No                         |

```
{  
  "name": "Teclado Mecánico",  
  "price": 85.99,  
  "stock": 25,  
  "categoryId": 1  
}
```

```
    "categoryId": 1  
  }  
}
```

#### Response 201 :

```
{  
  "id": 2,  
  "name": "Teclado Mecánico",  
  "price": 85.99,  
  "stock": 25,  
  "categoryId": 1,  
  "category": { "id": 1, "name": "Electrónica" }  
}
```

Protegido: JWT + Admin

---

#### PUT /products/:id

Actualiza un producto existente. Todos los campos son opcionales.

##### Request body:

| Campo      | Tipo          | Obligatorio |
|------------|---------------|-------------|
| name       | string        | No          |
| price      | number        | No          |
| stock      | number        | No          |
| categoryId | number   null | No          |

```
{  
  "price": 79.99,  
  "stock": 30  
}
```

#### Response 200 :

```
{  
  "id": 2,  
  "name": "Teclado Mecánico",  
  "price": 79.99,  
  "stock": 30,  
  "categoryId": 1,  
  "category": { "id": 1, "name": "Electrónica" }  
}
```

**Response 404 :** "Product not found"

Protegido: JWT + Admin

---

### **DELETE /products/:id**

Elimina un producto existente.

**Response 200 :** Devuelve el producto eliminado

```
{
  "id": 2,
  "name": "Teclado Mecánico",
  "price": 79.99,
  "stock": 30,
  "categoryId": 1,
  "category": { "id": 1, "name": "Electrónica" }
}
```

**Response 404 :** "Product not found"

Protegido: JWT + Admin

---

## **4.3 Categories**

### **GET /categories**

Lista todas las categorías ordenadas por nombre alfabéticamente.

**Response 200 :**

```
[
  { "id": 1, "name": "Electrónica" },
  { "id": 2, "name": "Ropa" },
  { "id": 3, "name": "Hogar" }
]
```

Protegido: JWT

---

### **GET /categories/:id**

Obtiene una categoría por su ID.



**Response 200 :**

```
{ "id": 1, "name": "Electrónica" }
```

**Response 404 :** "Category not found"

Protegido: JWT

---

## **POST /categories**

Crea una nueva categoría.

**Request body:**

| Campo | Tipo   | Obligatorio           |
|-------|--------|-----------------------|
| name  | string | Sí (1–128 caracteres) |

```
{ "name": "Deportes" }
```

**Response 201 :**

```
{ "id": 4, "name": "Deportes" }
```

**Response 409 :** "Category name already exists"

Protegido: JWT + Admin

---

## **PUT /categories/:id**

Actualiza el nombre de una categoría.

**Request body:**

```
{ "name": "Electrónica y Tecnología" }
```

**Response 200 :**

```
{ "id": 1, "name": "Electrónica y Tecnología" }
```

**Response 409 :** "Category name already exists"

Protegido: JWT + Admin

---

### DELETE /categories/:id

Elimina una categoría existente.

**Response 200** : Devuelve la categoría eliminada

```
{ "id": 1, "name": "Electrónica y Tecnología" }
```

**Response 404** : "Category not found"

Protegido: JWT + Admin

---

## 4.4 Users (admin)

### GET /users

Lista todos los usuarios registrados.

**Response 200** :

```
[
  { "id": "uuid-1", "email": "admin@mail.com", "role": "admin", "createdAt": "..."
  { "id": "uuid-2", "email": "user@mail.com", "role": "user", "createdAt": "..." }
]
```

Protegido: JWT + Admin

---

### PATCH /users/:id/role

Cambia el rol de un usuario. No se puede cambiar el propio rol ni degradar al único admin restante.

**Request body:**

| Campo | Tipo   | Valores          |
|-------|--------|------------------|
| role  | string | "user" o "admin" |

```
{ "role": "admin" }
```

**Response 200 :**

```
{ "id": "uuid-2", "email": "user@mail.com", "role": "admin", "createdAt": "..." }
```

**Response 403 :** "Cannot change your own role" o "Cannot demote the only admin"

Protegido: JWT + Admin

---

### **PATCH /users/me/password**

Cambia la contraseña del usuario autenticado.

**Request body:**

| Campo           | Tipo   | Obligatorio            |
|-----------------|--------|------------------------|
| currentPassword | string | Sí                     |
| newPassword     | string | Sí (mín. 8 caracteres) |

```
{  
  "currentPassword": "12345678",  
  "newPassword": "nuevaPass99"  
}
```

**Response 200 :**

```
{ "message": "Password updated" }
```

Protegido: JWT

---

### **PATCH /users/me/email**

Cambia el email del usuario autenticado.

**Request body:**

| Campo    | Tipo   | Obligatorio       |
|----------|--------|-------------------|
| newEmail | string | Sí (email válido) |
| password | string | Sí                |

```
{
  "newEmail": "nuevo@mail.com",
  "password": "12345678"
}
```

**Response 200 :**

```
{ "message": "Email updated" }
```

Protegido: JWT

## 4.5 Resumen rápido

| Método | Ruta            | Auth        | Notas                                                   |
|--------|-----------------|-------------|---------------------------------------------------------|
| POST   | /auth/register  | No          | Primer usuario = admin                                  |
| POST   | /auth/login     | No          |                                                         |
| GET    | /auth/me        | JWT         |                                                         |
| GET    | /products       | JWT         | Lista paginada con items[] , cada item incluye category |
| GET    | /products/:id   | JWT         | Incluye category completa                               |
| POST   | /products       | JWT + Admin |                                                         |
| PUT    | /products/:id   | JWT + Admin |                                                         |
| DELETE | /products/:id   | JWT + Admin |                                                         |
| GET    | /categories     | JWT         |                                                         |
| GET    | /categories/:id | JWT         |                                                         |
| POST   | /categories     | JWT + Admin |                                                         |
| PUT    | /categories/:id | JWT + Admin |                                                         |
| DELETE | /categories/:id | JWT + Admin |                                                         |
| GET    | /users          | JWT +       |                                                         |

|       |                    |             |                           |
|-------|--------------------|-------------|---------------------------|
|       |                    | Admin       |                           |
| PATCH | /users/:id/role    | JWT + Admin |                           |
| PATCH | /users/me/password | JWT         | Cambiar propia contraseña |
| PATCH | /users/me/email    | JWT         | Cambiar propio email      |

## 4.6 Tabla de códigos HTTP

| Código | Significado                                 |
|--------|---------------------------------------------|
| 200    | OK (GET, PUT, DELETE exitoso)               |
| 201    | Created (POST exitoso)                      |
| 400    | Bad Request (validación falló)              |
| 401    | Unauthorized (falta token o token inválido) |
| 403    | Forbidden (no tiene el rol requerido)       |
| 404    | Not Found (recurso no existe)               |
| 409    | Conflict (nombre duplicado)                 |